Project Design Phase
Solution Architecture

Date	20/10/2026
Team ID	
Project Name	i-Movies — Movie Ticket Booking System
Maximum Marks	4 Marks

Solution Architecture:

i-Movies follows a three-layer full-stack architecture consisting of a React-based presentation layer, an Express API layer, and a MongoDB data layer. The frontend and backend are deployed separately: the client is hosted on Vercel, while the Node.js/Express server can run on a Node hosting service such as Render. The frontend communicates with the backend through HTTP requests to REST-style API routes.

1. Presentation Layer — React and Vite

The client is built with React and Vite, with React Router handling navigation between customer and administrator pages. Tailwind CSS is used for interface styling, and shared context manages application state such as authentication, theme, and notifications. Movie pages let customers browse and search movies, filter by genre, sort results, view movie details, explore theatres and showtimes, inspect seat availability, create or cancel bookings, manage favourites, and submit reviews. The Vercel rewrite configuration routes direct visits and page refreshes back to the single-page application entry point.

2. API and Application Layer — Node.js and Express

The backend runs on Node.js with Express 5. The server starts by loading configuration and connecting to MongoDB, then mounts API route groups under the /api prefix. Routes direct incoming requests to controllers, which validate input and perform the relevant application operation through Mongoose models. Middleware handles authentication, rate limiting, and error processing. The API exposes a health endpoint and separate route groups for authentication, profiles, movies, reviews, theatres, shows, bookings, favourites, and administrator operations.

3. Data Layer — MongoDB and Mongoose

MongoDB stores the application's persistent records, with Mongoose schemas defining the data structures. The main models include User, Admin, Movie, Theatre, Show, Booking, Review, and Favorite. These records support account management, movie and theatre catalogues, show schedules, seat states, booking status, reviews, and saved movies. Passwords are hashed with bcryptjs rather than stored as plain text.

4. Authentication, Authorization, and API Security

Customer and administrator sign-in use JSON Web Tokens (JWT). The client stores their tokens separately in browser local storage and attaches the appropriate bearer token to API requests. Backend middleware verifies tokens and checks whether the request requires a customer or administrator. Protected operations restrict access to permitted records; for example, customers can access their own bookings, while administrator routes provide broader management access. Login and registration routes are rate-limited.

5. Booking and Seat Availability Flow

A customer selects a show and requests available seats. The backend validates the requested seat identifiers and attempts to reserve only seats that remain available, using a conditional update to avoid two requests booking the same seat. After seat reservation, the backend creates the booking record. If booking creation fails, it attempts to release the reserved seats. When a booking is cancelled, the cancellation status is recorded before seats are released, and seat release checks that each seat still belongs to that booking.

6. Deployment and Frontend–Backend Communication

The frontend uses the VITE_API_BASE_URL setting to determine the backend API address, including the /api prefix. The backend uses MONGO_URI for database connectivity, JWT_SECRET for signing tokens, and CORS_ORIGINS to specify the exact browser origins permitted to call the API. Since the client and server are deployed separately, the deployed frontend origin must be listed in CORS_ORIGINS, and the client must be rebuilt after changing its Vite environment variable. The /api/health endpoint can be used to check backend availability.

Example — Solution Architecture Diagram:

Customer / Admin		**Browser UI**
React + Vite React Router / Tailwind	→	Vercel-hosted client
Shared API service VITE_API_BASE_URL	**HTTP**	Express 5 API Routes / Controllers / Middleware
JWT bearer token	→	Auth / role checks Rate limiting / validation
MongoDB via Mongoose	↔	User, Admin, Movie, Theatre, Show, Booking, Review, Favorite